YOUR TECHNOLOGY & INNOVATION PARTNER

Gitlab CI/CD Session 3

Agenda

1. Running test in pipeline
2. Code coverage
3. Code Quality Integration with SonarQube

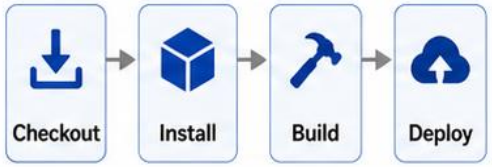
YOUR TECHNOLOGY & INNOVATION PARTNER

Running test in pipeline

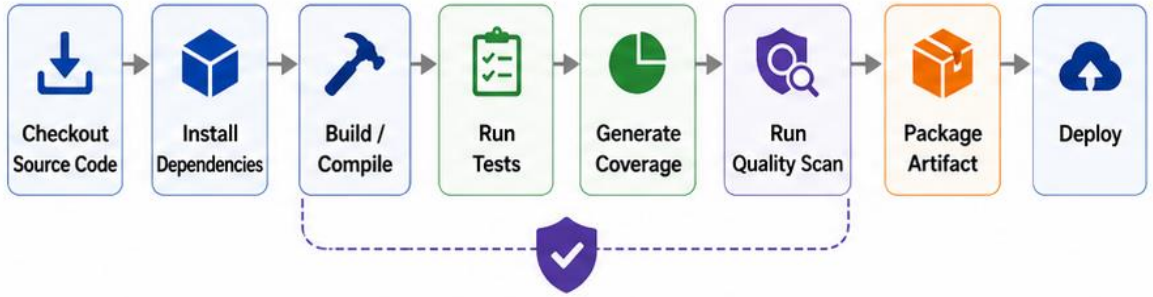
Pipeline Flow

1. TYPICAL VIEW

A common assumption is that a pipeline only builds and deploys.



2. STANDARD FLOW



Validation happens before release.

3. WHY TEST EARLY

- If tests fail, the pipeline should stop.
- This prevents unnecessary build or deploy steps.
- Early feedback helps developers fix issues faster.

4. FAIL FAST BENEFITS

Save Time

Stop the pipeline early when validation fails.

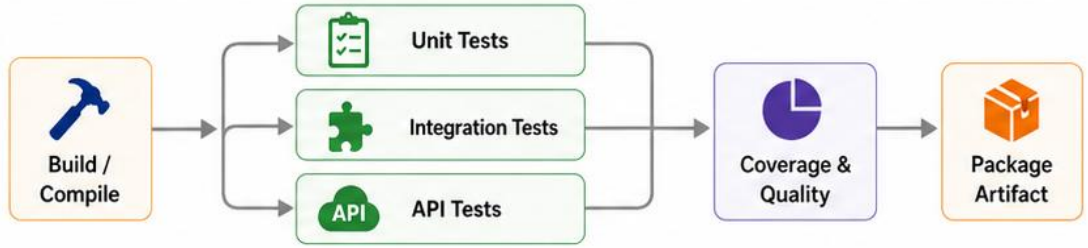
Save Runner Resources

Avoid wasting compute on later stages.

Faster Feedback

Developers learn about failures sooner.

5. PARALLEL TEST JOBS



In many projects, tests run in parallel to reduce pipeline time.

6. KEY MESSAGE

Run tests as early as possible to save time, reduce waste, and speed up feedback.

Test Automatic Flow

Automated testing in CI/CD helps teams catch issues earlier and apply the same validation standard to every change.

1. MANUAL TESTING

If tests are only run manually, teams often face these problems:

- Forgot to run tests
- Different local environments
- Works on my machine
- Inconsistent results
- Time-consuming

2. AUTOMATED FLOW

Push Code → Pipeline Starts → Run Tests → Test Report → Pass / Fail Result

Every push or merge request can trigger the same validation process.

3. WHY IT MATTERS

- Developers may forget to test before pushing code.
- Different machines can produce different results.
- Manual testing does not scale well on large projects.
- Automation provides faster and more consistent feedback.

4. TEAM BENEFITS

Automatic Checks
Tests run on every push or merge request.

Same Standard
All developers follow the same validation rules.

Earlier Detection
Issues are found before merge or release.

Safer Refactoring
Teams gain more confidence when changing code.

5. WHAT THE PIPELINE PROVIDES

1. Runs tests automatically
2. Applies consistent validation
3. Blocks broken code from moving forward
4. Gives reviewers better visibility
5. Reduces human error
6. Protects source code quality

6. KEY MESSAGE

A good pipeline does not rely on people remembering to test — it makes testing automatic, consistent, and reliable.

Test Types

1. TEST STRATEGY

Not every pipeline runs every type of test. Teams should choose the right tests for the right stage.

2. COMMON TEST TYPES

Unit Tests
Check small functions, classes, or modules. Fast, stable, and easy to automate.

Integration Tests
Verify how multiple modules or services work together.

API Tests
Validate endpoints, status codes, requests, responses, and contracts.

UI / E2E Tests
Test user flows through the interface. Slower but valuable before release.

Lint Checks
Check code style, formatting, syntax rules, and conventions.

Security Tests
Detect dependency vulnerabilities, secret leaks, or security issues.

3. WHEN TO RUN THEM

- **Unit:** every push / every merge request
- **Integration:** merge request or main branch
- **API:** merge request or main branch
- **UI / E2E:** main branch, release branch, or nightly
- **Lint:** every push
- **Security:** scheduled scan, merge request, or release pipeline

4. QUICK VS SLOW

Fast Feedback
- Unit Tests
- Lint Checks

Broader Validation
- Integration Tests
- API Tests
- UI / E2E Tests
- Security Tests

5. EXAMPLE PIPELINE CHOICE

Push Code → Lint + Unit → Integration + API → UI / E2E → Security / Release Checks

Teams balance speed and coverage by splitting test types across pipeline stages.

Test Stage

1. STAGE STRUCTURE

A good test stage has a clear structure and separate jobs.

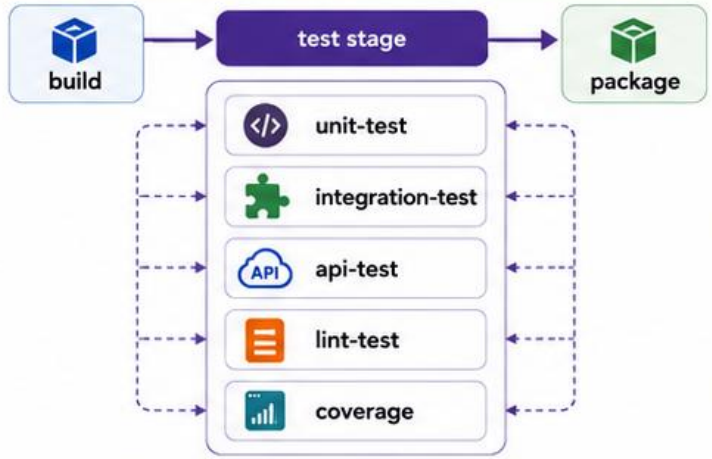
unit-test | integration-test | api-test | lint-test | coverage

- Each job should have a single responsibility.
- Avoid one huge test job that is hard to debug and optimize.

2. EXAMPLE PIPELINE STAGE

```yaml
stages:
  - build
  - test
  - package

unit-test:
  stage: test
integration-test:
  stage: test
api-test:
  stage: test
lint-test:
  stage: test
coverage:
  stage: test
```



Split the test stage into focused jobs for better speed and visibility.

3. WHY SPLIT JOBS

- Easier to debug when a specific job fails.
- Faster pipelines through parallel execution.
- Clear ownership for each test type.
- Easier to optimize slow jobs.

4. STAGE REQUIREMENTS

Auto Run	Job Fails	Pipeline Fails	Reports	Artifacts	Logs
tests start automatically.	failed tests fail the job.	failed jobs stop the pipeline.	clear test reports are available.	results can be reviewed later.	enough detail for debugging.

5. DEVELOPER VALUE

1. Find the failing job quickly
2. Read reports and logs
3. Understand the issue faster
4. Fix the problem with less guesswork

The goal is not only pass/fail, but faster troubleshooting.

Test Reports

1. WHY REPORTS MATTER

Test reports help teams understand more than just pass/fail.

- Total tests
- Passed
- Failed
- Failed cases
- Duration
- Error details

2. EXAMPLE TEST REPORT

Total Tests	Passed	Failed	Skipped	Duration
128	123	5	0	2m 14s

Failed Tests

Test Name	Failure Reason	File	Line
auth.service.spec.ts	should reject invalid password	auth.service.spec.ts	45
order.api.test.ts	returns 500 on timeout	order.api.test.ts	78
PaymentControllerTest	should create refund	PaymentControllerTest.java	132

Error Message

```
Expected status 401 but received 200
```

A good report helps teams quickly see what failed and why.

3. COMMON FORMAT

JUnit XML is a common format supported by many CI/CD tools.

JUnit XML

- Easy to parse
- Shown in pipeline UI
- Useful in merge requests
- Good for automation

4. TOOLS THAT GENERATE REPORTS

Jest
Can generate JUnit reports for JavaScript / TypeScript

Maven Surefire
Creates test reports for Java projects

Pytest
Can export JUnit XML for Python tests

5. TEAM BENEFITS

1. Find failing tests faster
2. Review results in merge requests
3. Debug issues with clearer details
4. Make code review more professional

Saving reports improves traceability and troubleshooting.

Artifactory

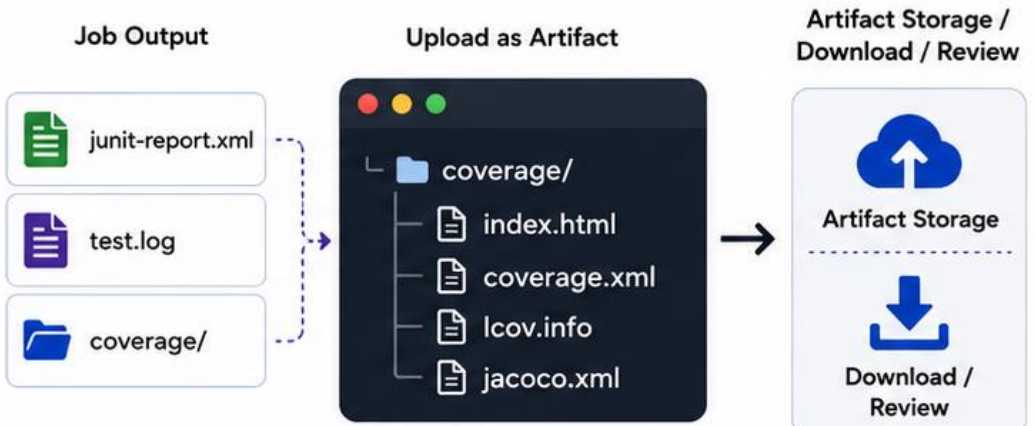
1. WHAT ARTIFACTS ARE

Artifacts are saved outputs from a job.

- Test Report
- Coverage Report
- Log File
- HTML Report
- XML Report

A pipeline can keep important files after the job ends.

2. EXAMPLE ARTIFACT



Users can download or review saved reports later.

3. WHY IT HELPS

- Useful when tests fail
- Better than scanning long terminal logs
- Easy to download and review
- Helps debugging and code review

4. COMMON ARTIFACTS

Test Reports	Coverage Reports	Logs	HTML Reports	XML Reports
See which tests failed	Check coverage details	Read execution output	Open rich browser reports	Used by CI/CD tools

5. PIPELINE EXAMPLE

Artifacts preserve useful outputs from the test stage.

Artifactory

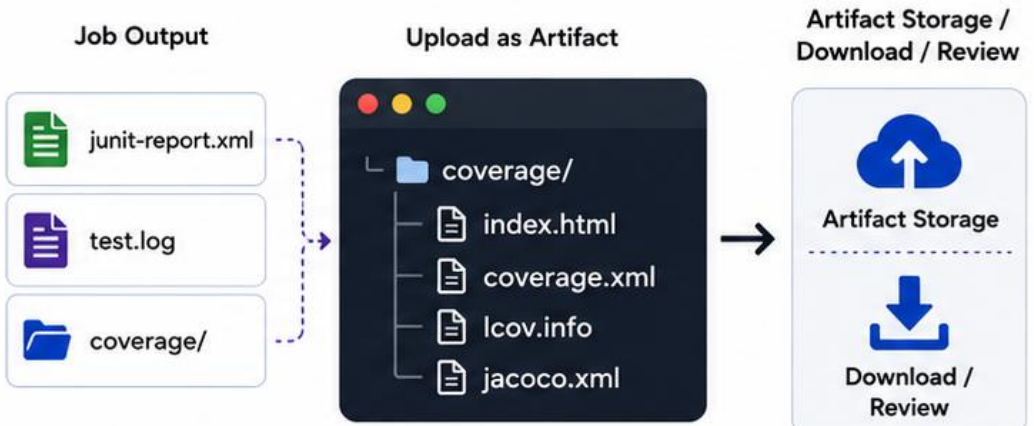
1. WHAT ARTIFACTS ARE

Artifacts are saved outputs from a job.

- Test Report
- Coverage Report
- Log File
- HTML Report
- XML Report

A pipeline can keep important files after the job ends.

2. EXAMPLE ARTIFACT



Users can download or review saved reports later.

3. WHY IT HELPS

- Useful when tests fail
- Better than scanning long terminal logs
- Easy to download and review
- Helps debugging and code review

4. COMMON ARTIFACTS

Test Reports	Coverage Reports	Logs	HTML Reports	XML Reports
See which tests failed	Check coverage details	Read execution output	Open rich browser reports	Used by CI/CD tools

5. PIPELINE EXAMPLE

Artifacts preserve useful outputs from the test stage.

Failure Fast

1. WHAT FAIL FAST MEANS

Fail fast means stopping the pipeline as soon as a critical issue is found, instead of continuing with unnecessary later steps.

Late stop (wasted effort)

Build → Test → Package → Deploy

Too late! Wasted time and resources.

Fail fast (stop early)

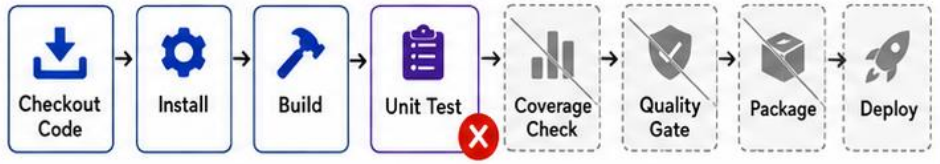
Build → Test → STOP

Stop here. Save time and resources.

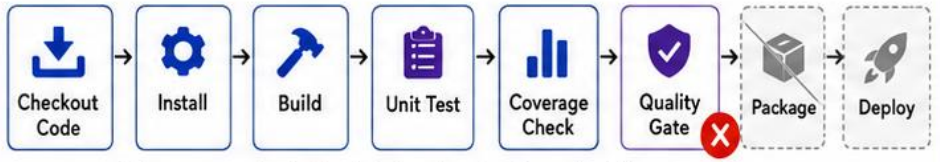
Stop early to save time and runner resources.

2. EARLY STOP EXAMPLE

Failure at Unit Test (stops immediately)



Failure at Quality Gate (stops immediately)



When a required check fails, the pipeline should not continue.

3. FAIL CONDITIONS

- Build failure
- Unit test failure
- Integration test failure in required environments
- Coverage below agreed threshold
- SonarQube Quality Gate failure
- Critical vulnerability found
- Severe lint error

4. BENEFITS

Save Time

Avoid running later stages unnecessarily

Save Runner Resources

Reduce wasted compute

Faster Feedback

Developers learn about issues sooner

Safer Delivery

Broken code does not move forward

5. WARNING VS ERROR

Warning

- Minor code smell
- Small style issue
- Non-blocking recommendation

Can be reported without failing the pipeline.

Error

- Broken build
- Failed required tests
- Critical security issue
- Failed Quality Gate

Should fail the pipeline.

A good pipeline is strict enough to protect quality, but not so rigid that it blocks normal development for every minor issue.

YOUR TECHNOLOGY & INNOVATION PARTNER

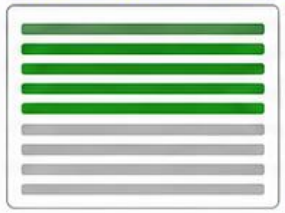
Code Coverage

Coverage Basics

1. WHAT COVERAGE MEANS

Coverage shows the percentage of code exercised by tests.

Project: 1,000 lines of code



750 lines executed by tests → Line Coverage = **75%**

- How much code is tested?
- Which parts are still untested?
- Did new code get tests?

2. COVERAGE EXAMPLE

Total Lines	Covered Lines	Uncovered Lines	Line Coverage
1,000	750	250	75%

```
1  function calculateTotal(items) {
2     let total = 0;
3     for (const item of items) {
4        if (item.price > 0) {
5           total += item.price;
6     } else {
7        total += 0;
8     }
  }
```

Executed (covered)

Not executed (uncovered)

Tests executed 75% of the code paths in this example.

3. COMMON TYPES

Line Coverage
How many lines were executed

Branch Coverage
How many if/else branches were executed

Function Coverage
How many functions were called

Statement Coverage
How many statements were executed

4. WHAT COVERAGE HELPS ANSWER

- How much of the codebase is exercised by tests?
- Which areas are not tested yet?
- Does new code have tests?
- Did coverage drop after merge?

5. IMPORTANT LIMITATION

Useful

- Finds untested areas
- Shows test reach
- Helps track trends

Not Enough Alone

- High coverage can still miss bugs
- Coverage does not replace assertions
- Coverage does not replace code review

Coverage is a signal, not a guarantee.

Coverage Mindset

1. COMMON MISUNDERSTANDING

A common mistake is assuming that higher coverage always means higher quality.

90%

High number ≠ guaranteed quality

- High coverage can still miss bugs
- Weak assertions reduce test value
- A beautiful number alone is not the goal

2. TWO EXAMPLES

Case A: 90% Coverage

Many lines executed, but weak assertions.

90%

Bug can still slip through

Case B: 60% Coverage

Focused on critical business logic.

60%

Often more valuable

3. WHAT COVERAGE DOES NOT REPLACE

- Code Review
- Test Case Design
- Strong Assertions
- Static Analysis
- Good Unit Tests

Coverage is one signal, not the whole quality strategy.

4. WHAT COVERAGE IS GOOD FOR

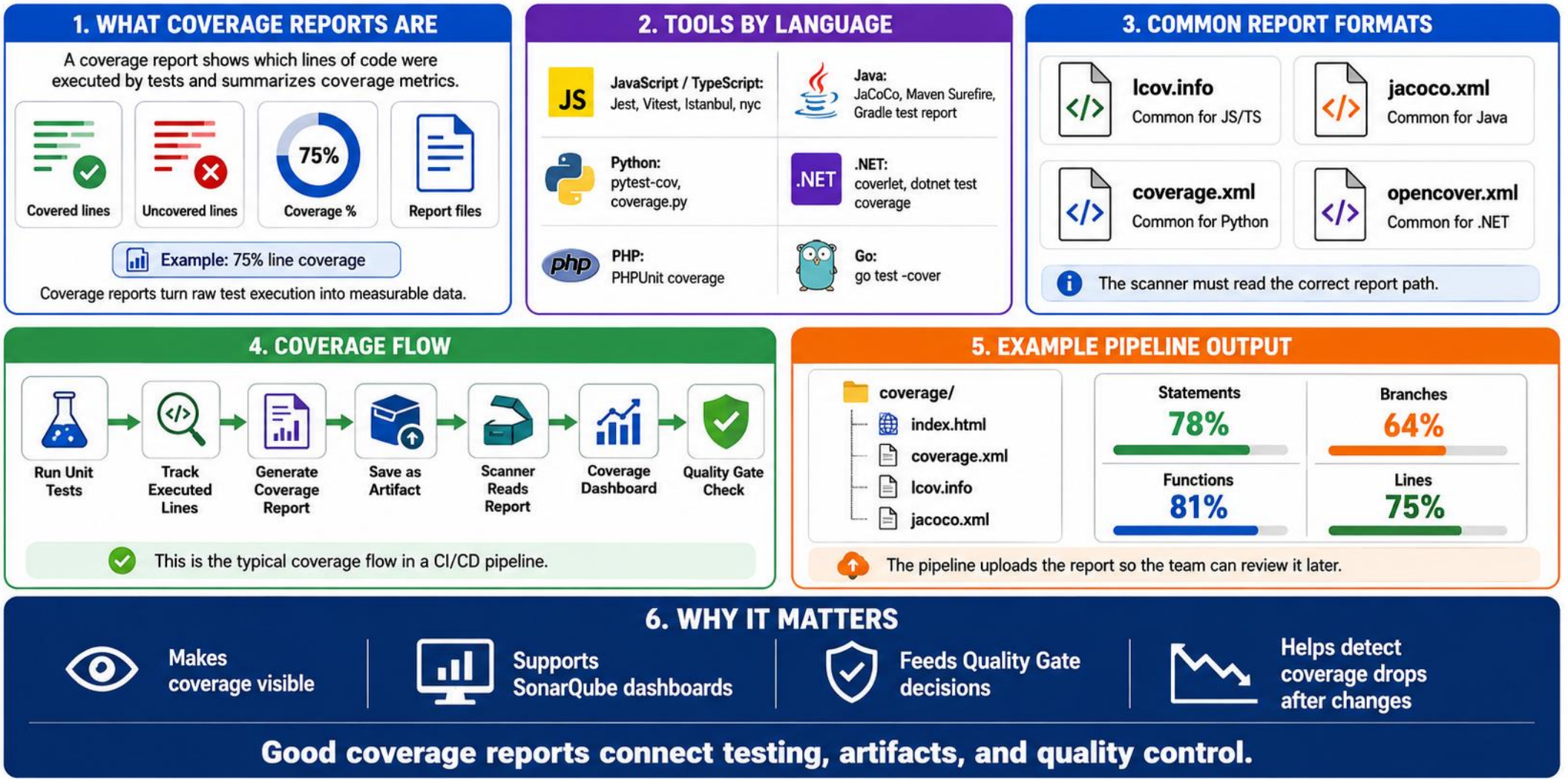
- Finds untested areas
- Highlights risky gaps
- Tracks trends over time
- Shows whether new code has tests

5. BETTER MINDSET

- Use coverage to reduce software risk
- Focus on important business logic
- Review test quality, not only percentages
- Combine coverage with review and static analysis
- Aim for meaningful validation, not just a higher number

The goal is not a pretty number — the goal is lower risk.

Coverage Reports



Code Quality

1. WHAT CODE QUALITY MEANS

Code quality ensures the codebase is easy to work with, safe to change, and dependable over time.

- Readable
- Maintainable
- Reliable
- Secure

Good code is easier to understand, change, test, and trust.

2. RUNS ≠ GOOD CODE

Runs successfully ≠ Still has quality problems

- Long functions
- Too many responsibilities
- Complex logic
- Duplicated code
- Unclear variable names
- Poor exception handling
- Hard-coded secrets
- Hidden bugs
- Security vulnerabilities

3. WHY AUTOMATE IT

- Find issues earlier
- Reduce manual review effort
- Keep coding standards consistent
- Catch risky code before release
- Prevent production incidents

4. COMMON QUALITY CHECKS

- Code Smells
- Complexity
- Duplication
- Bug Patterns
- Security Issues

Modern CI/CD pipelines should check quality automatically.

5. EXAMPLE TOOLS

SonarQube	ESLint	Checkstyle	PMD	Pylint
overall code quality platform	JavaScript / TypeScript linting	Java style rules	static analysis for code issues	Python code analysis

SonarQube is a common platform to aggregate quality signals and enforce rules in CI/CD.

6. QUALITY FLOW

1. **Commit Code** — Push changes to repository
2. **Run Lint / Static Analysis** — Analyze code for issues automatically
3. **Generate Quality Report** — Produce issues, metrics, and trends
4. **Review in Dashboard** — Inspect results and identify problems
5. **Quality Gate / Decision** — Pass or fail build based on rules

Code quality should be checked automatically, early, and continuously.

Code Quality

1. WHAT CODE QUALITY MEANS

Code quality ensures the codebase is easy to work with, safe to change, and dependable over time.

- Readable
- Maintainable
- Reliable
- Secure

Good code is easier to understand, change, test, and trust.

2. RUNS ≠ GOOD CODE

Runs successfully ≠ Still has quality problems

- Long functions
- Too many responsibilities
- Complex logic
- Duplicated code
- Unclear variable names
- Poor exception handling
- Hard-coded secrets
- Hidden bugs
- Security vulnerabilities

3. WHY AUTOMATE IT

- Find issues earlier
- Reduce manual review effort
- Keep coding standards consistent
- Catch risky code before release
- Prevent production incidents

4. COMMON QUALITY CHECKS

- Code Smells
- Complexity
- Duplication
- Bug Patterns
- Security Issues

Modern CI/CD pipelines should check quality automatically.

5. EXAMPLE TOOLS

SonarQube	ESLint	Checkstyle	PMD	Pylint
overall code quality platform	JavaScript / TypeScript linting	Java style rules	static analysis for code issues	Python code analysis

SonarQube is a common platform to aggregate quality signals and enforce rules in CI/CD.

6. QUALITY FLOW

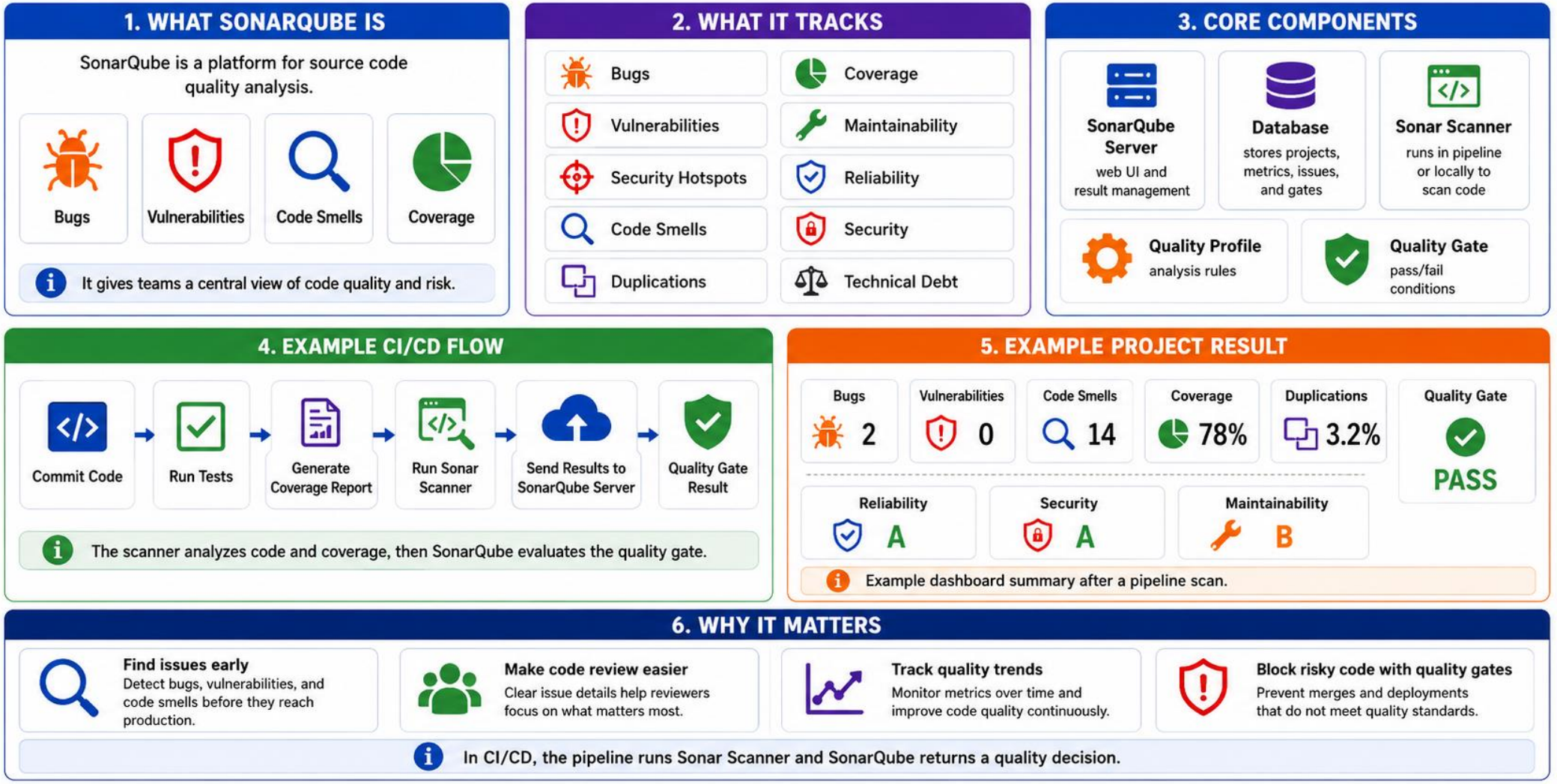
1. **Commit Code** — Push changes to repository
2. **Run Lint / Static Analysis** — Analyze code for issues automatically
3. **Generate Quality Report** — Produce issues, metrics, and trends
4. **Review in Dashboard** — Inspect results and identify problems
5. **Quality Gate / Decision** — Pass or fail build based on rules

Code quality should be checked automatically, early, and continuously.

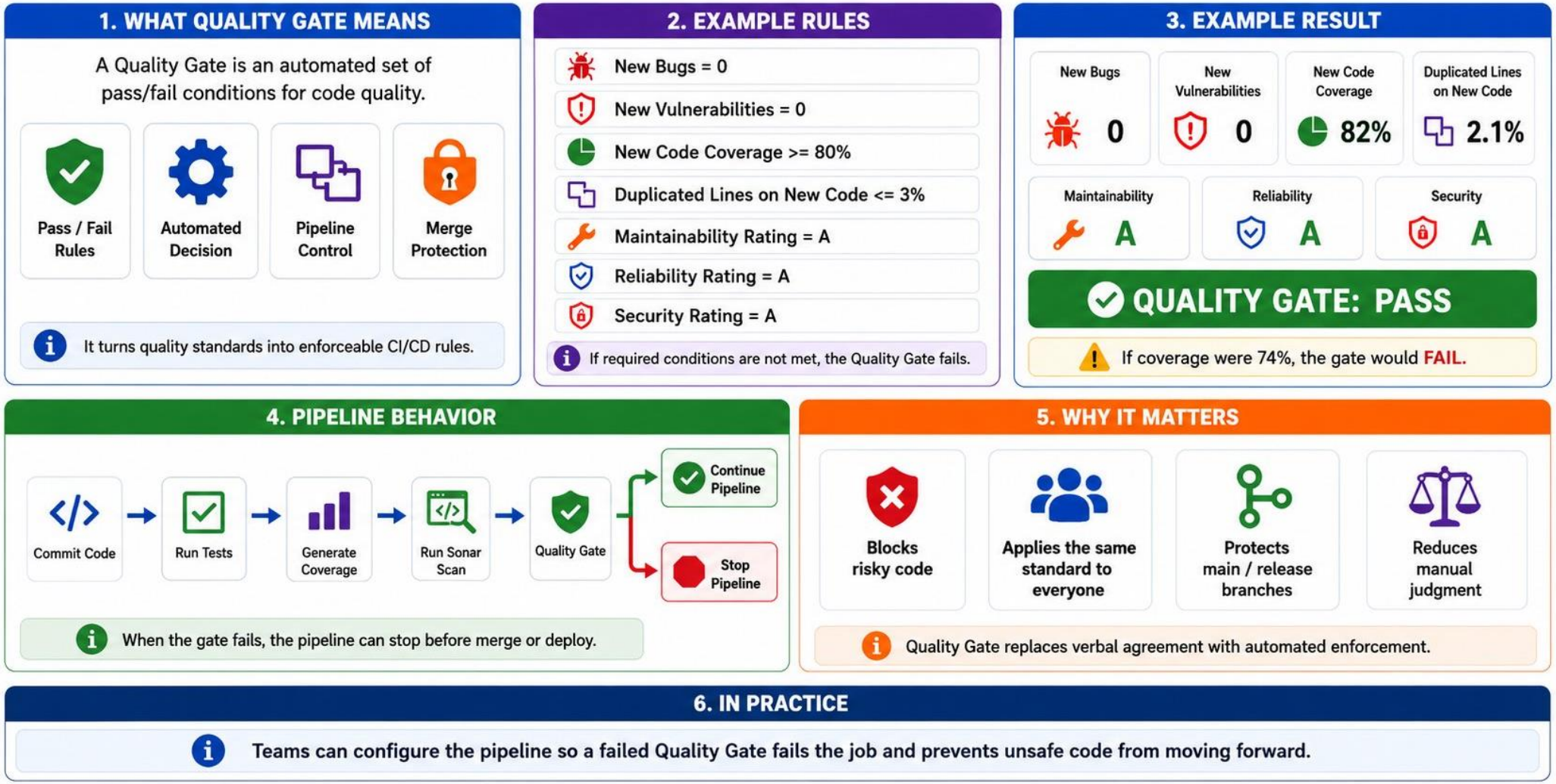
YOUR TECHNOLOGY & INNOVATION PARTNER

Code Quality Integration With SonarQube

Sonarqube


1. WHAT SONARQUBE IS
SonarQube is a platform for source code quality analysis.
Bugs
Vulnerabilities
Code Smells
Coverage
It gives teams a central view of code quality and risk.
2. WHAT IT TRACKS
Bugs
Vulnerabilities
Security Hotspots
Code Smells
Duplications
Coverage
Maintainability
Reliability
Security
Technical Debt
3. CORE COMPONENTS
SonarQube Server
web UI and result management
Database
stores projects, metrics, issues, and gates
Sonar Scanner
runs in pipeline or locally to scan code
Quality Profile
analysis rules
Quality Gate
pass/fail conditions
4. EXAMPLE CI/CD FLOW
Commit Code
Run Tests
Generate Coverage Report
Run Sonar Scanner
Send Results to SonarQube Server
Quality Gate Result
The scanner analyzes code and coverage, then SonarQube evaluates the quality gate.
5. EXAMPLE PROJECT RESULT
Bugs 2
Vulnerabilities 0
Code Smells 14
Coverage 78%
Duplications 3.2%
Quality Gate PASS
Reliability A
Security A
Maintainability B
Example dashboard summary after a pipeline scan.
6. WHY IT MATTERS
Find issues early
Detect bugs, vulnerabilities, and code smells before they reach production.
Make code review easier
Clear issue details help reviewers focus on what matters most.
Track quality trends
Monitor metrics over time and improve code quality continuously.
Block risky code with quality gates
Prevent merges and deployments that do not meet quality standards.
In CI/CD, the pipeline runs Sonar Scanner and SonarQube returns a quality decision.

Quality Gate



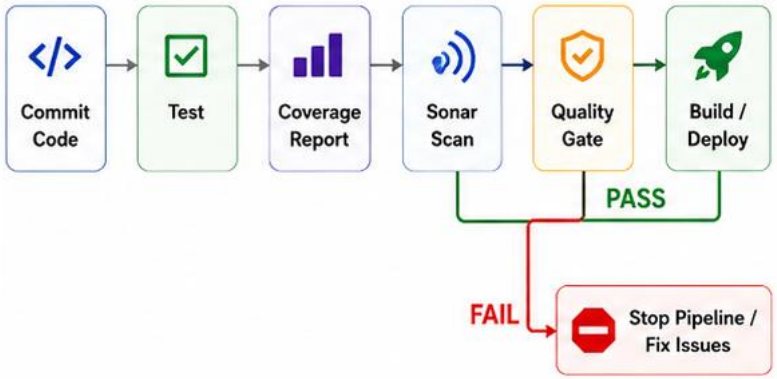
19. SONAR INTEGRATION

A typical SonarQube integration in CI/CD runs tests first, generates coverage, scans the code, and then waits for the Quality Gate result.

1. INTEGRATION FLOW

1. Developer pushes code or opens merge request
2. Pipeline installs dependencies
3. Run unit tests
4. Generate coverage report
5. Run Sonar Scanner
6. Send results to SonarQube Server
7. SonarQube analyzes code and coverage
8. Evaluate Quality Gate
9. Pipeline continues or stops

2. EXAMPLE PIPELINE FLOW



3. REQUIRED CONFIG

- SONAR_HOST_URL
- SONAR_TOKEN
- sonar.projectKey
- sonar.sources
- sonar.tests
- sonar.exclusions
- sonar.coverage.exclusions
- coverage report path

Do not hard-code tokens in the repository.
Store them in CI/CD variables or a secret manager.

4. EXAMPLE CONFIG

```
sonar.projectKey=my-app
sonar.sources=src
sonar.tests=tests
sonar.exclusions=dist/**, node_modules/**
sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

CI VARIABLES

SONAR_HOST_URL

SONAR_TOKEN

5. EXAMPLE RESULT

Bugs	Vulnerabilities	Code Smells
1	0	9

Coverage	Duplications
81%	2.4%

QUALITY GATE: PASS

6. WHY IT MATTERS

- Automates quality checks
- Combines test + coverage + static analysis
- Blocks risky code before merge
- Gives developers clear feedback

Integrating SonarQube ensures consistent code quality and reduces technical debt.

YOUR TECHNOLOGY & INNOVATION PARTNER

Demo

THANK YOU!